CIS 5050: Software Systems

Team: T04
Members:
Vinay Padegal, Navya Saxena, Ashay Katre, Adrija Mukherjee

Description of the design components-wise:

1. Key-value store

1.2 Design

The design and architecture of the key-value store is broadly as follows:

- A single coordinator server runs at 127.0.0.1:10000.
- There are 6 workers running from 127.0.0.1:10001 to 127.0.0.1:10006.
- There are 6 tablets in total. Tablet i ($1 \leq i \leq 6$) is replicated across workers i, (i+1) mod 6, and (i+2) mod 6. For example, tablet 2 is on workers 2, 3, and 4, with worker 2 as the primary. Tablet 6 is on workers 6, 1, and 2, with worker 6 as the primary.
- All communication with the frontend as well as among the servers within the backend is done using gRPC.

1.3 Coordinator

The coordinator, or master server, is responsible for tracking which tablets are stored on which workers, identifying the current primary for each tablet, mapping each row to a certain tablet, monitoring the liveness of workers, and redirecting client requests to the appropriate worker.

The coordinator launches a background thread that monitors worker heartbeats every 3 seconds and updates their liveness status. This information is then used to identify the current primary for each tablet, support the Admin Console, and more.

The coordinator implements a server using gRPC which offers the following APIs:

- **Lookup(*row*)**: This function is called by the client to determine which worker currently holds the primary copy of the tablet that contains the specified row.
- **Heartbeat(*worker_id, timestamp*)**: This function is called by the workers to send a heartbeat to the coordinator.
- **WorkerLookup(*worker_id*)**: This function is called by the workers when they recover from a crash to know who the current primary is for each of their tablets.
- **Alive(*worker_id*)**: This function returns the current liveness status (up/down) of a given worker.

1.4 Worker

The workers are the servers that handle most of the operations and are responsible for storing the actual data. All data—whether text or binary—is stored as a string, since C++ strings can hold arbitrary byte sequences, including null bytes, making them suitable for storing both textual and binary data.

Each worker holds 3 tablets in memory, which are accessed to serve client requests. Every update from a client is replicated to the secondary and tertiary replicas of the tablet and also logged to disk. Furthermore, the full in-memory tablet is serialised and checkpointed to disk after every 10 update operations.
The worker implements a gRPC service called KeyValueStore which exposes the following APIs:

- **Get(*row*, *column*)**: This function indexes the appropriate tablet and returns the value corresponding to the (*row*, *column*) pair. If the *row* or the *column* is not available, it returns an error.
- **Put(*row*, *column*, *value*)**: This function indexes the appropriate tablet and writes *value* in (*row*, *column*) key pair. It also calls ReplicatePut on the other replicas.
- **CPut(*row*, *column*, *current_value*, *new_value*)**: This function indexes the appropriate tablet and writes *value* in (*row*, *column*) key pair. If the *row* or *column* is not available or the current value is not equal to *current_value*, it returns an error. It also calls ReplicateCPut on the other replicas.
- **Delete(*row*, *column*)**: This function indexes the appropriate tablet and deletes the value corresponding to the (*row*, *column*) pair. If the *row* has no more entries, the *row* is deleted as well. It also calls ReplicateDelete on the other replicas.
- **ReplicatePut(*row*, *column*, *value*)**: This function is identical to the Put request but without the replication functionality.
- **ReplicateCPut(*row*, *column*, *current_value*, *new_value*)**: This function is identical to the CPut request but without the replication functionality.
- **ReplicateDelete(*row*, *column*)**: This function is identical to the Delete request but without the replication functionality.
- **CopyTablet(*tablet_id*)**: This function is used to serialise and transfer a tablet to a secondary worker that is recovering from a crash in order to get it back 'in sync'. The serialised tablet is sent across in 1MB chunks to facilitate larger data transfers.
- **AdminConsole(*password*, *command*)**: This function sets admin_workerStatus=false if it receives a SHUTDOWN request from the Admin Console. It recovers all tablets and sets admin_workerStatus=true if it receives a RESTART request.
- **Crash recovery demo** LINK

2. Frontend HTTP Servers

- **Design and Features Implemented**: Frontend HTTP servers are multithreaded C++/pthreads processes that expose a single, path-based interface supporting HEAD, GET and POST methods with chunked transfer encoding for efficient file uploads and downloads. We maintain persistent keep-alive connections until the client explicitly closes or logs out, and we secure sessions by generating unique, random cookies on login and storing each session_id -> username mapping in the KV-store under

"session-ids," which keeps all three frontends in sync. Every operation - user account management, email, storage - is routed through this single server, returning appropriate HTTP status codes (200, 401, 503, etc.). A HEAD /health endpoint lets the load balancer and admin console probe each server's liveness, and SIGINT handling ensures a clean shutdown across all threads.
Session ids are stored in the KV store in the following manner:

	active-users	session id 1	session id 2
session-ids	{<user 1>:<session id 1>}, {<user 2>:<session id 2>}	user 1	user 2

The load balancer runs on port 8080 that handles each new client by returning a 307 Temporary Redirect to one of three frontend servers (ports 8081–8083). Backends are chosen in round-robin fashion, but only if they respond successfully to a HEAD /health probe; otherwise they're skipped. After issuing the redirect, the balancer closes the connection so all subsequent traffic goes directly to the chosen frontend.

- **Major Challenges**
    - Integrating user account, email service, storage service and admin console operations through http server while different team members worked on different functionalities was challenging. We integrated one service at a time and were able to resolve merge conflicts successfully.
    - We originally tried using non-blocking TCP connections for health checks, but discovered this interfered with our worker nodes' port bindings. Switching instead to simple blocking HTTP HEAD requests resolved the issue and reliably reports backend liveness.

3. User Account Service

- User Account Service provides /signup, /credentials (login), and /changepassword endpoints on the frontend, each backed by a gRPC UserAccountService that stores credentials in the KV store under <username>-userinfo. Signup returns a 409 if the user exists, otherwise creates the user. Login validates credentials and issues a unique session cookie via generateSessionCookie(). Password change verifies existence and old password before atomically updating the KV store with a CPut, returning appropriate errors on failure. Using KV store conditional puts ensures atomicity and consistency across concurrent account operations.
User information is stored in KV store in the following manner:

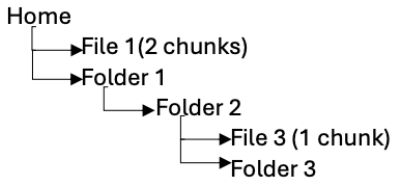
	password
<username>-userinfo	<password set by user>

4. Storage Service

- **Design and implementation**: Storage Service uses a single KV-store row <username>-storage to maintain a hierarchical file system: the special column metadata lists root entries, while each folder "node" is marked by a column <absolute path> = KV-Folder and its child list in <absolute path>-metadata. Files are similarly tagged with <absolute path> = KV-File, and their content is sharded into 1 MB

chunks—each chunk stored under <absolute path>-<chunk#>-content and tracked in <absolute path>-metadata. This layout enables fast listing (by reading only sentinel columns) and supports all operations—download, rename, move, delete—for both files and folders.

For example, if we have the following hierarchy:



KV store row will look like this:

	metadata	File 1	File 1-metadata	File 1-1-content	Folder 1	Folder 1-metadata	Folder1/Folder2	Folder1/Folder2-metadata	Folder1/Folder2/File 3	Folder1/Folder2/File 3-metadata	Folder1/Folder2/File 3-1-content	Folder1/Folder2/Folder 3	Folder1/Folder2/Folder 3-metadata
<username>-storage	File 1,Folder 1	KV-File	File 1-1-content	Byte array	KV-Folder	Folder1/Folder2	KV-Folder	Folder1/Folder2/File 3,Folder1/Folder2/Folder 3	KV-File	Folder1/Folder2/File 3-1-content	byte array	KV-Folder	

- **Features**:
  - **Listing**
    The storage service presents a hierarchical view by reading only the metadata columns in the user's "<username>-storage" row. For the current folder (or root), it fetches the appropriate metadata column - either "metadata" or "<folder-path>-metadata" - which contains a comma-separated list of child names (files and subfolders). It parses that list and displays each entry without retrieving its content, allowing users to see available files and folders efficiently.
  - **Deletion**
    File Deletion involves three steps: removing the sentinel marker ("KV-File"), deleting its metadata column ("<path>-metadata") and all associated chunk-content columns ("<path>-<chunk#>-content"), and then updating the parent folder's metadata list by removing the file name via a conditional put. Folder Deletion is recursive: the service enumerates all columns whose keys begin with "<folder>/", deletes each file or subfolder using the file deletion procedure, then removes the folder's own sentinel and metadata column. Finally, it updates the parent folder's metadata to remove the folder entry, ensuring the entire subtree is expunged atomically.
  - **Renaming**
    Renaming a file (or folder) first writes a new sentinel at the target path and copies over its metadata/chunk columns to the new key, then deletes the old sentinel, metadata, and chunk columns. A conditional put updates the containing folder's metadata list by replacing the old name with the new one. *Folder Rename* applies the same pattern recursively: all descendant columns under the old folder prefix are reinserted under the new prefix before the old columns are removed, and the parent metadata list is updated to reflect the new folder name.
  - **Moving**
    Moving is implemented as a cross-folder rename. For files, the service copies the

sentinel, metadata, and chunk columns from the source folder to the destination folder's prefix, deletes the old columns, removes the file name from the source folder's metadata, and adds it to the target folder's metadata via conditional puts. *Folder Move* performs the same recursive copy-then-delete on all descendants and updates both source and destination parent metadata lists to ensure the folder appears only under its new parent.

- **Challenges**:
  Implementing reliable multipart parsing over persistent HTTP connections was a key challenge: browsers would hang waiting for more bytes until we robustly detected the final multipart boundary (--<boundary>--) in the incoming buffer, including handling multiple boundaries when optional folder fields were present.

4. Webmail service

Aims to mimic key functionality of real-world email clients (e.g., Gmail), enabling users to view inbox and sent mail, compose, send, reply to, forward, and delete emails, including external domains. The EmailService is implemented as a modular class, exposes simple HTTP endpoints to users. Internally, it delegates all data handling to a distributed KVstore accessed using gRPC.

Data Storage Strategy

KV-store rows back each user's inbox and sent folders:

- inbox_email:<username>
- sent_email:<username>

Each email is stored as a column, where:

- **Column name** = encrypt(metadata) (e.g., reversed string of sender, recipient, subject)
- **Value** = email body (text content)

Maintain a **meta** column in each row that contains a comma-separated list of encrypted column.

Metadata is a semicolon-separated string:
sender:<alice@localhost>;recipient:<bob@localhost>;subject:<Meeting>;timestamp:<2025-05-01 14:00:00>;
This string is reversed (simple placeholder for encryption) to generate a unique, non-guessable column key. (In organizational setting, a more advanced encryption eg:- complex hashing mechanism)

Why Our Design Is Optimal

Our design optimizes for the most frequent operation in a mail service: inbox rendering. By storing a single encrypted metadata string as the column name and keeping the metadata list in a lightweight meta column, we avoid extra reads, reduce data duplication, and maintain fast, constant-time access to full email headers during inbox rendering. This design is significantly more space-efficient and lookup-optimized than storing each metadata field (e.g., subject, sender) as separate columns per email.

Forward/Reply Implementation

- We chose a simple and stateless strategy for reply and forward: instead of maintaining explicit message chains or threading metadata, we generate a new standalone message where the original content is quoted inside the body. allows users to contextually respond to prior emails while preserving our simple metadata-based structure. It also avoids additional cross-references, keeping KV store access fast and flat.

- When replying or forwarding, the service fetches the original metadata and body using the encrypted ID. It prepends the body with a quoted version of the original and modifies the subject (Re: or Fwd: prefix). The composed message is then stored just like a new email

SMTP Server

- To fully support external email delivery and test end-to-end interoperability with real-world clients like Gmail or Thunderbird, we implemented a custom SMTP server - allows us to to relay mail externally using DNS MX records, the functionality of real MTA.
- It speaks the full SMTP protocol, supporting commands like HELO, MAIL FROM, RCPT TO, DATA, and QUIT. Thunderbird or other clients can be configured to send mail to our server (on port 2500), which will deliver it to local inboxes or externally via MX resolution.

External Delivery via MX Lookup

- When enabled with the -x flag, the server performs DNS MX lookups for domains like gmail.com or seas.upenn.edu It connects to the resolved mail host on port 25 and relays the message using a standard SMTP dialogue

Design

- Local recipients (@localhost) are stored directly into our KV store as inbox entries
- External recipients are relayed using real DNS lookups and TCP connections
- rewrite MAIL FROM:<user@localhost> to user@seas.upenn.edu before external delivery to reduce spam filtering
- We enforce RFC 5322 compliance: only one Date: header, valid Message-ID, and correct header-body separation

5. Admin console

The Admin Console provides a web-based interface for monitoring and controlling both frontend and backend servers in a distributed system. It connects to a gRPC-based master node and six worker nodes using `MasterNodeClient` and `KeyValueStoreClient` wrappers. Through gRPC calls, it can check server liveness (`Alive` RPC) and trigger administrative actions like `shutdown` or `restart` using the `AdminConsole` method. Frontend services (running on ports 8081–8083) are monitored and controlled via socket-based health checks and shell command invocations (e.g., `pkill`, relaunch). The `fetchWorkerStatuses()` and `checkHttpAlive()` functions aggregate worker and frontend statuses in JSON, while the `home()` method renders them in an HTML dashboard with toggle buttons. This setup enables centralized, real-time system control and visibility via a lightweight and extensible interface.

Contributions:

Vinay Padegal: Key-Value Store, Admin Console
Navya Saxena: Front End with load balancer, User Accounts, Storage Service
Ashay Katre: Email Service, Admin Console
Adrija Mukherjee: Storage Service